AIRBNB PRICE PREDICTION

FINAL REPORT

Jessie Dicker and Norma Arredondo

DATA-4950-D01

Dr. Yanfang Liu

May 3, 2026

I. Executive Summary

This analysis aimed to predict the price of Airbnbs in New York City using listing data. Accurately predicting price is challenging yet critical for Airbnb hosts, as it directly impacts their property demand, competitiveness, and revenue. Understanding the key drivers of price, as this analysis aimed to do, allows hosts to make more informed and strategic pricing decisions.

Our data-driven approach used Airbnb listing data from 2019 which included important pricing features such as location, unit size, and characteristics. We used this data to train several different regression models to identify the method with the most reliable predictions: Linear Regression, Ridge Regression, Lasso Regression, Decision Tree, Random Forest, and Gradient Boosting. We used a 5-fold cross-validation method to identify the best model settings and tune their complexity.

The majority of the linear models performed similarly, with test metrics around 0.40. The more complex models were better able to capture the relationship in the data. The best-performing model was our tuned Gradient Boosting model, which explained about 47.6% of the variability in nightly Airbnb prices. The model's predictions were off by around $76.46 on average. We chose this model ultimately due to its balance between predictive accuracy and overfitting, even though the tuned Random Forest was able to achieve the highest test R^2.

While the modeling results indicate only moderate predictive accuracy, we are still able to gain actionable insights into the pricing behaviors of the Airbnbs when looking at the most important features identified by the Gradient Boosting model. We found that location and unit types were the greatest drivers of price, as multiple location-based features were identified. Thus,

pricing strategies should be implemented based on these variables. By prioritizing these factors, hosts can better align their listings with the market and maximize profits.

II. Problem Statement and Description

This regression analysis focuses on predicting the nightly price of an Airbnb listing in New York City. The host faced challenges in determining the price they should charge the travelers based on the amenities and features of the rental place. Travelers would want to rent a place that has all their necessities, and it is affordable. So, how much should they charge? With this problem, we built a predictive model to help hosts set prices for their listings. Our predictive model also determines if specific features influence the price of the listing. A wrong predictive model comes with real-world consequences. If the host decides to overcharge the properties, their listing would not be booked, and will lose out on income. If the host decides to undervalue the properties, they lose out on income. In the end, the host loses income. Which is why it is important to have an accurate predictive model.

The source of the dataset comes from Kaggle. It contained 48,895 rows and 16 columns. Each row represents Airbnb listings in New York City in 2019. The columns represent features such as geographic coordinates, neighborhood and its identifiers, review counts and frequency, room type, days of availability per year, and minimum nights requirements. The target variable for this regression project is the price of the listing. The price range is from $0 to $10,000. We explored both the target variables and the features.

Out of all 16 features in the dataset, we decided to drop five features. The five features dropped for data leakage are ID, Name, Host ID, Host's Name, and Last Review. The ID feature

was an arbitrary numeric identifier. The value of this column is not a predictive value and doesn't determine the price. The names are unstructured name titles of the listing. The host ID feature was the host's arbitrary numeric identifier. The host's name is an unstructured feature. The last review feature leaks temporal information about listing activity. All these data leakage features are not predictors that could determine the price, and are unnecessary to keep in the dataset.

Figure 1

Airbnb Prices Distribution

Note: Prices are in USD. There are counts in $2,000 to $10,000

III. Exploratory Data Analysis

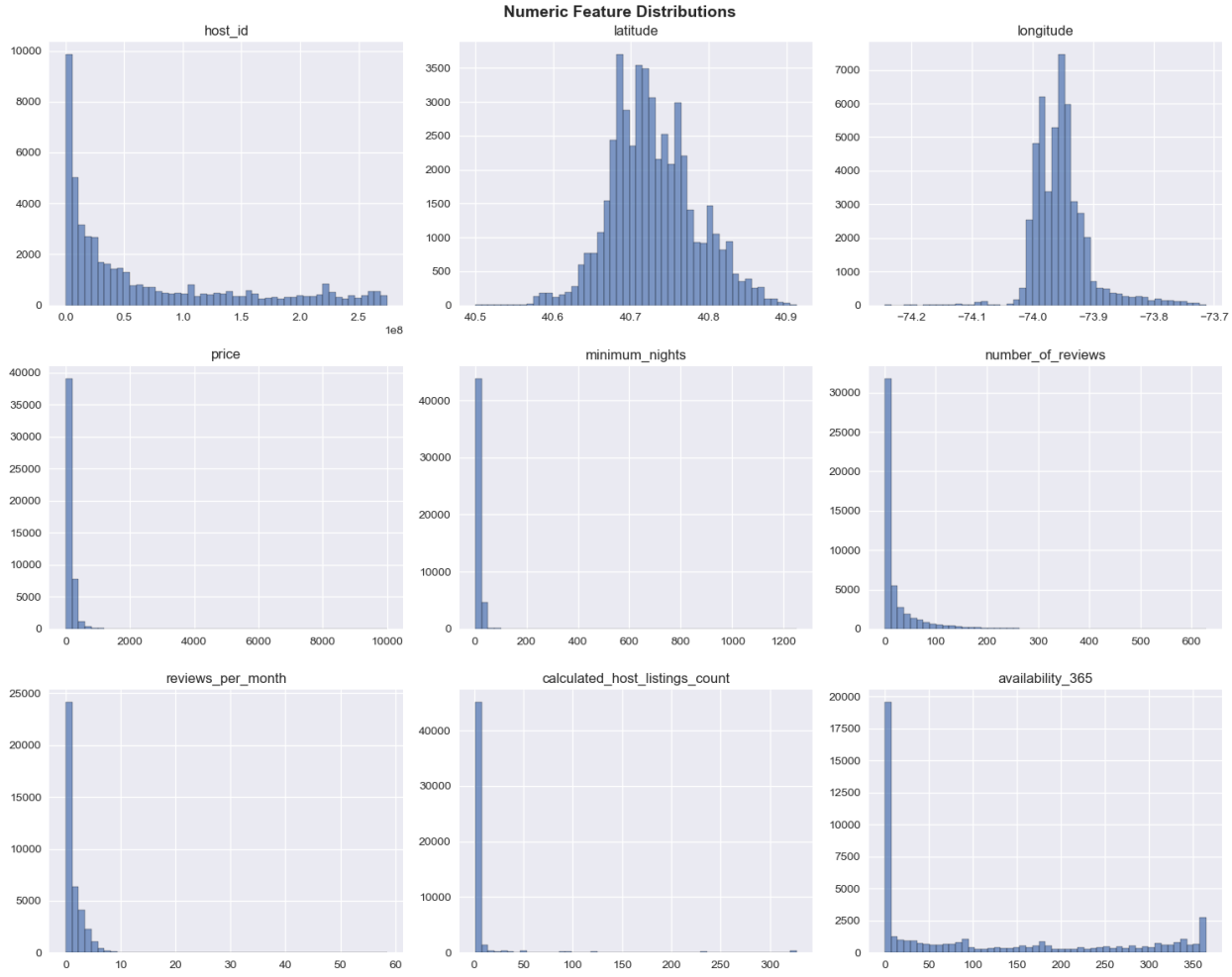
The Exploratory Data Analysis explores the raw dataset and sets the area we need to focus on. There are five important findings during our Exploratory Data Analysis.

1. Our target variable was heavily right-skewed, as seen in **Figure 1**. The maximum price value for a listing was $10,000. The mean of the nightly price was $152.72, which exceeded the median of $106.00. This indicated that the number of maximum price values shifted the average higher. This would later be fixed.
2. Many numeric features are revealed to be right-skewed. **In Figure 2**, the histogram shows numeric features such as Host ID, Minimum Nights, Number of Reviews, Reviews per Month, Calculated Host Listings Count, and Availability are heavily skewed right. With this information, there were many outliers across features in the raw dataset. This is the finding before data leakage.
3. Our strongest numeric predictor for price was Longitude. We compared all the numeric features with price and found that Longitude has the strongest correlation with price at -.150 (the absolute correlation). The longitude reflects the East and West geography of New York City, which means the North and South (latitude) don't have as much effect on the price as the longitude.
4. Many numeric features had weak correlations with price. Although longitude has the strongest correlation with price, many numeric features show weak correlation. For example, the correlation between the number of availability and price is 0.082. All numeric features show a weak relationship with price. This means that feature interactions, rather than linear relationships, could determine the price. You can look at **Figure 3** to see the correlation values of the numerical features.

5. The reviews per month and the number of reviews were collinear. Longitude was the strongest predictor for price, but there are two features that have the strongest correlation with each other. The features are the Reviews per Month and the Number of Views. The correlation of those two is 0.550. This multicollinearity was a concern because both features could compete for the same signal in linear models.

Figure 2

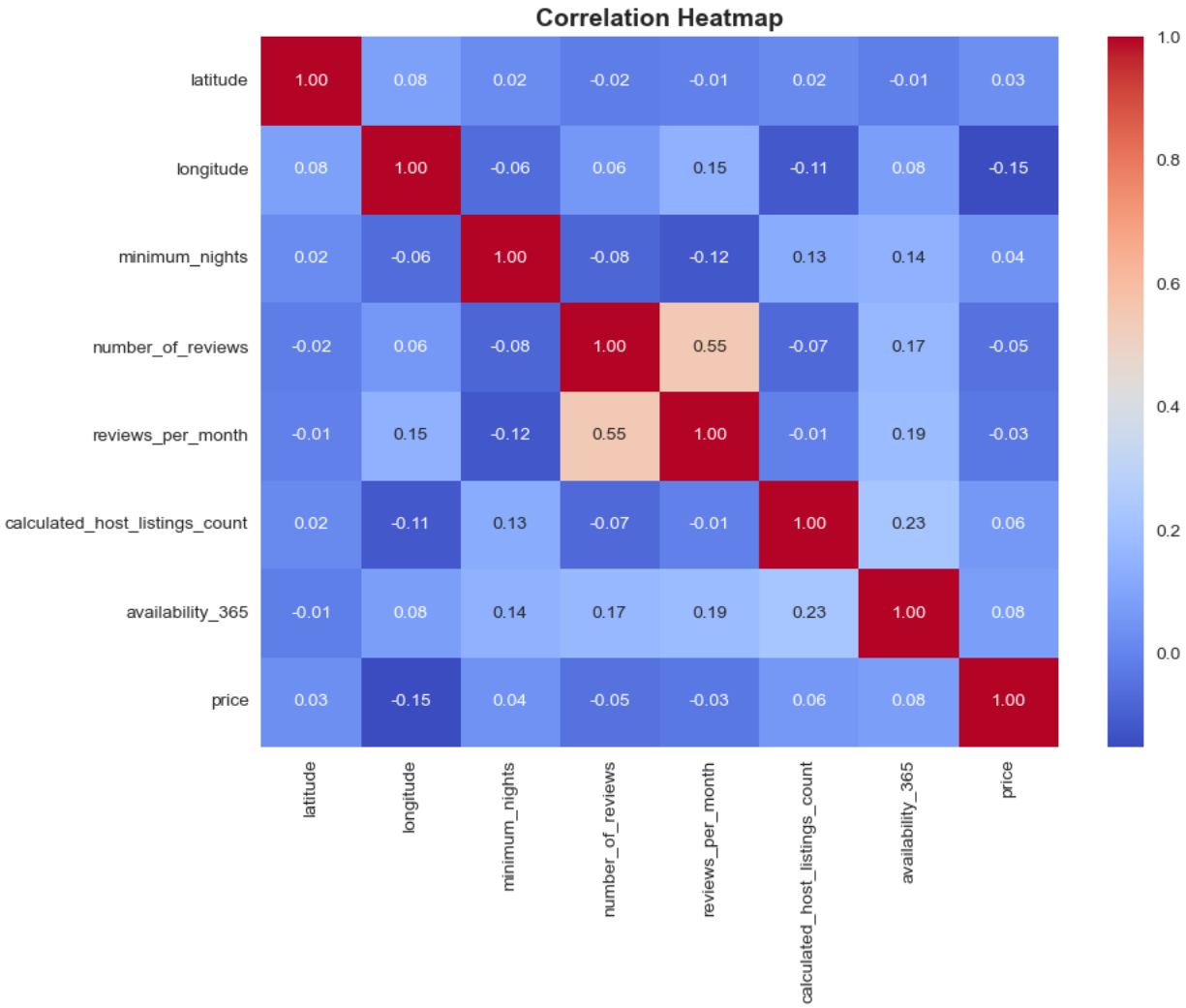
All Numeric Features Distributions



Note: Host ID, Latitude, Longitude, Price, Minimum Nights, Number of Reviews, Reviews per Month, Calculated Host Listing Count, and Availability are the numerical features.

Figure 3

All numeric features and their Correlation



Note: The blues are the weaker correlations and the reds are higher correlations

IV. Data Preprocessing

During the preprocessing, we clean the dataset before the train and test split to maintain dataset consistency. The simple cleaning involved outliers and data leakage. Outliers and data leakage should be addressed to prevent misleading predictions. There were no missing values in our target variable, price. Nor were there duplicate values in the other features. The only things we looked out for in price were outliers and unrealistic prices. **Table 1** shows the columns that were cleaned and the reasoning.

Table 1
Columns that were cleaned

Column	% Affected	Decision	Reasoning
Price	0.02%	Dropped when price is $0	Listing prices at zero is unrealistic
Price	1.00%	Capped at 99th percentile	Extreme values ($10,000) would dominate a regression loss function
Reviews per Month	20.50% each for both train and test set	Imputed with 0 where number of reviews is 0	Missing values correspond to listings with no reviews

Minimum Nights	13.45%	Winsorized using IQR bounds from training data	Extreme high value requirements are unlikely and can distort model training
Calculated host listings count	14.45%	Winsorized using IQR bounds from training set	To avoid model bias from a rare large portfolio

We want to note that not all features with high outlier values were capped. Since longitude and latitude are geographic coordinates, the outliers were not changed because the coordinates represent actual location, and capping them would disrupt the spatial relationship. The number of reviews and reviews per month were not capped or removed because high review counts could indicate listing popularity, so modifying them would remove meaningful signals. We had to remove prices equal to zero and capped the extreme values to maintain a meaningful price range. The maximum price decreased from $10,000 to $799 since the 99th percentile was at $799. The $799 price is more reasonable than the $10,000 listing. There were 7,868 missing values in the training data and 1,959 in the test data for Reviews per Month. Imputing Reviews per Month was necessary as it could distort the relationship between the two review features. After this, there were no more missing values in our data.

After cleaning before split, the dataset contained 48,884 rows and 11 columns. We used the clean dataset to split 80/20. 80% into a training set, which makes 38,728 rows in training data. And 20% on the test set, which makes 9,682 rows in the test data. Since the target variable is continuous, stratification was not used.

The final training size after all cleaning and split ratio was 38,728 rows and 10 columns.

V. Feature Engineering

Feature Engineering involves making new features. Since the existing numeric features show a weak correlation with price, we created new features to improve the relationship. **Table 2** shows five new features we constructed from existing columns with the technique and rationale.

Table 2

Engineered Features by Technique

New Feature Name	Technique	Rationale
Log minimum nights	Log Transformation (log1p)	To reduce the right skew and stabilizes variance
Distance to Times Square	Haversine Distance Formula (Location Based)	To convert coordinates into kilometers to measure distance to Time Square
Location Tier	Binning	To capture the price impact of location
Is New Listing	Binary Flag	Zeros could mean new listing or hasn't been booked
Review Density	Ratio	To show effectiveness of the density rather than number of reviews

These were constructed to see improvement in our relationship with price, but two engineered features outperformed the existing columns. The first one is the Distance to Times Square, for which we calculated the longitude and latitude using the Haversine Distance formula. Then the Location Tier, where we binned the new feature, Distance to Times Square. The Location Tier was separated into four distance zones: Very Central (0-2 km), Central (2-5 km), Outer (5-10 km), and Remote (10+ km). These two location-based engineered features. The Distance to Times Square had an absolute correlation with the price of 0.334. The Location Tier had an absolute correlation of 0.369. Both outperformed the longitude correlation of 0.150 and the latitude correlation of 0.033. The interpretable distance and tier captured more of the pricing signal than existing coordinates.

After constructing new engineered features, there were some columns that needed to be encoded. It was necessary to convert the categorical variable into a numerical variable to contribute to the predictive modeling. There are four categorical columns that we have encoded. The first encoding column was Room Type. We used Ordinal Encoding to capture the hierarchy of private room, shared room, and home or apartment. The second encoding column was Location Tier, which we used Ordinal Encoding. We used Ordinal Encoding because the distance tier has a natural ordering. Then we encoded Neighborhood Grouped. For this, we used One-Hot Encoding since there is no natural ordering of the borough categories. It was later dropped as we One-Hot encoded the neighborhood since it is unordered neighborhood categories.

Once we finished encoding, we scaled eleven continuous features using the Standard Scaler fit only on the training set. The eleven columns that were scaled are Longitude, Latitude, Distance to Times Square, Location Tier, Minimum Nights, Log Minimum Nights, Number of

Reviews, Reviews per Month, Review Density, Host Listing Count, and Availability. We applied standardization to the data to use in the Ridge, Lasso, and linear regression models. The binary dummy variables, such as Neighborhood and Is New Listing, were excluded from scaling.

After dropping leakage columns, the dataset had 10 features. Then we constructed 5 new engineered features. The Neighborhood Grouped was dropped after Neighborhood dummy variables were created, since it captured the geographic pricing gradient more precisely. We dropped Minimum Nights since the Log Minimum Nights reduced the skew. We also dropped Location Tier instead of Distance to Times Square because the continuous distance measurement is more precise for modeling purposes. Finally, after one-hot encoding the Neighborhood Grouped, it added 24 dummy variables and other categories. In total, the final feature count is 36 columns for both the training and test sets.

VI. Modeling Approach and Results

We chose to include an array of models in order to identify the best performing model for our particular problem and our data. We used a Linear Regression model as a baseline because it is the simplest possible model, one which can be compared to more complex models. Next, we used a Ridge Regression model as it is able to handle multicollinearity and overfitting in linear models using an L2 regularization. For the final linear model, we used a Lasso model, because it can perform feature selection by shrinking coefficients to zero. We then used more complex models to evaluate the need to capture non-linear relationships. First, we used a Decision Tree model because it can capture these non-linear relationships as well as feature interactions. A Random Forest model was used to reduce the overfitting in the Decision Tree while still capturing complex patterns. Finally, a Gradient Boosting model was fit to further capture these

complex patterns while correcting previous mistakes in building trees. The Linear Regression model was fit using scaled and unscaled data to evaluate the impact of scaling. All other linear models as well as the Gradient Boosting model were fit using scaled data, while the tree-based models were fit using unscaled data.

The results from these models are contained in **Table 3**:

Table 3

Modeling Results

Model	Train R^2	Test R^2	RMSE
Random Forest	0.928	0.474	75.91
Gradient Boosting	0.508	0.467	76.46
Decision Tree	0.564	0.389	81.82
Ridge Regression (Scaled)	0.401	0.385	82.12
Linear Regression	0.401	0.385	82.12
Linear Regression (Scaled)	0.401	0.385	82.12
Lasso Regression (Scaled)	0.401	0.385	82.13

All of our linear models obtained similar results. There was little improvement in the scaled versus unscaled models, which indicates that scaling the data did not lead to much meaningful improvement in the model's ability to capture the relationship in the data. The tree-based models show a noticeable improvement when compared to the linear models, indicating that the relationship between predictors and target is not a simple, linear relationship but instead is more complex. The Random Forest model was able to achieve the highest train and test R^2, at 0.928 and 0.474 respectively. Our worst-performing model was the Lasso regression model using

scaled data, with a train R^2 of 0.401, a test R^2 of 0.385, and an RMSE of 82.13. These were very similar results to the rest of the linear models, all of which underfit the data.

Looking at the distance between train and test R^2 allowed us to evaluate if the model was overfitting. We determined an overfitting threshold of 0.05; that is, if the gap between train and test metrics is greater than this threshold, the model is likely overfitting. By this calculation, the Random Forest and Decision Tree models both exhibited signs of overfitting. The gap between train and test R^2 in the Random Forest model was 0.453, indicating severe overfitting. This is likely due to the fact that the model was too complex for the signal in the data. Instead of capturing the real relationship, it likely learned noise in the training set that it was unable to generalize to the new data. The Decision Tree model also shows signs of moderate overfitting, with its R^2 gap being 0.175.

After obtaining our modeling results, we used 5-fold cross validation to further evaluate model performance across different subsets of the data. The results are contained in **Table 4**:

Table 4

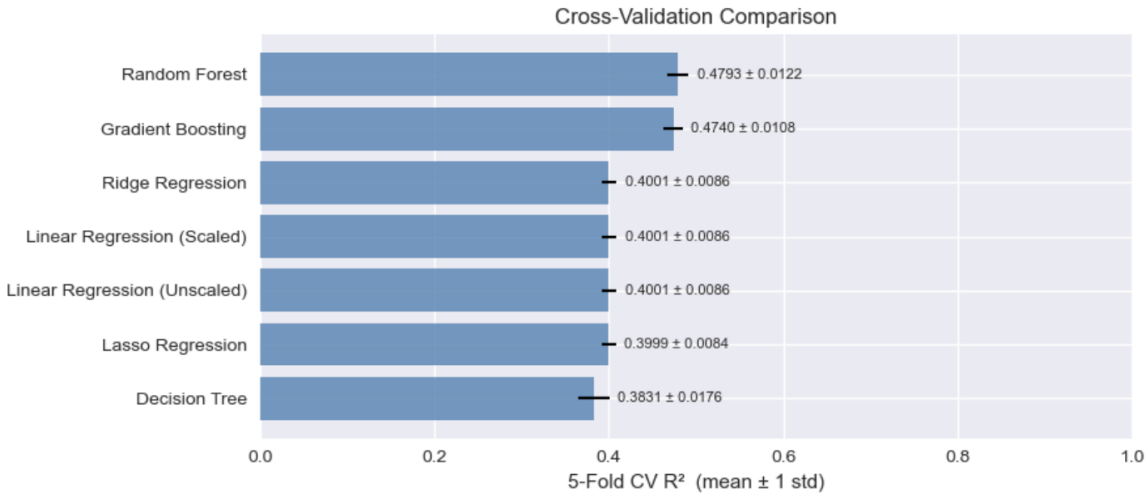
5-Fold Cross Validation Results

Model	CV Mean R^2	CV std
Random Forest	0.479	0.012
Gradient Boosting	0.474	0.011
Linear Regression	0.400	0.009
Linear Regression (Scaled)	0.400	0.009
Ridge Regression (Scaled)	0.400	0.009
Lasso Regression (Scaled)	0.3999	0.008
Decision Tree	0.383	0.018

All models performed similarly to the initial evaluation in the cross-validation. Their mean R^2 were close to the initial metrics obtained. The Random Forest and Gradient Boosting models achieved the highest average cross-validated R^2 (0.479 and 0.474, respectively). The linear models once again achieved nearly identical performance, providing further evidence of their underfitting. The Decision Tree model performed the worst overall, with the highest standard deviation (0.018) and lowest CV mean R^2, indicating high variability across folds. None of the models had a standard deviation above 0.03, indicating that they are all fairly stable.

Figure 4

5-Fold Cross-Validation Results Bar Chart



We also tuned the hyperparameters of the Ridge Regression, Random Forest, and Gradient Boosting models using cross-validation. The new metrics from the tuned models are contained in **Table 5**:

Table 5

Tuned Model Results

Model	Train R^2	Test R^2	Improvement from Untuned Model
Tuned Random Forest	0.744	0.498	0.023
Tuned Gradient Boosting	0.542	0.476	0.009
Tuned Ridge Regression	0.401	0.385	0.000

The only model that was not improved by hyperparameter tuning was the Ridge Regression model. There was no difference between the original and tuned metrics, indicating that regularization was not a factor in model performance. The Gradient Boosting and Random Forest model improved, indicating that these models benefitted from controlling their complexity.

VII. Model Selection and Interpretation

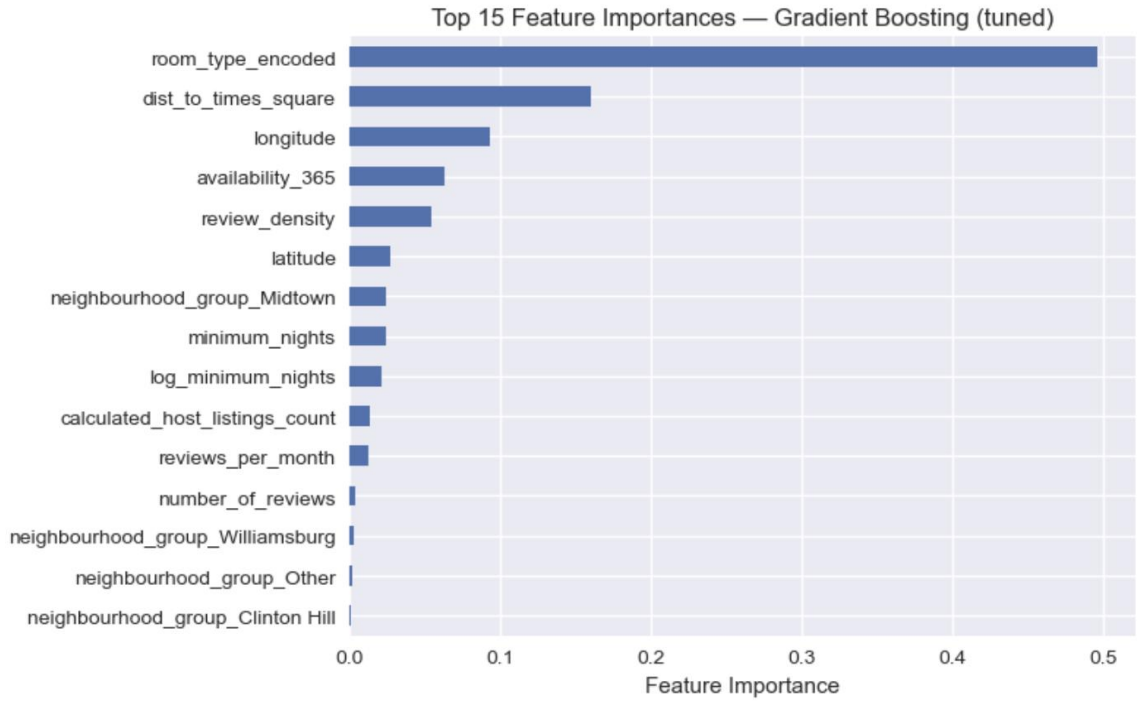
We selected the tuned Gradient Boosting model as our best. This model had the second-highest test R^2 (0.476) combined with a small gap between train and test metrics (0.066). This provides the best balance between overall accuracy and overfitting. The average cross-validation test R^2 was close to other metrics achieved in the analysis, at 0.474, and the small standard deviation (0.011) indicates that the model is stable. Compared to the default simple Linear Regression model with a test R^2 of 0.385, the tuned Gradient Boosting model was able to improve upon this baseline metric by +0.091.

The tuned Random Forest model achieved the highest test R^2 at 0.498, but the large gap between train and test metrics (0.246) suggests a significant amount of overfitting even in the tuned model. For these reasons, we chose the Gradient Boosting model as it achieves almost as high of a test metric without the risk of overfitting.

Our tuned Gradient Boosting model was able to identify the top 15 features by their importance, shown in **Figure 5**. The top 3 most important features in the dataset were the type of room, the distance to Times Square, and the longitude. Knowing these important features allows us significant insight into what drives the Airbnb pricing. The room type feature measures what kind of rental the Airbnb is – shared room, private room, or full unit. In a larger sense, this feature is indicative of *who* is renting the units, whether it's a single person, group, or a couple. This would directly tie to the price of the unit, as larger private units would likely be more expensive. The distance to Times Square feature is an engineered feature that measures the distance to one of New York City's most central, famous locations. This feature is predictive of price because it is directly tied to the demand for the unit. More central locations with close proximity to popular tourist destinations, such as Times Square, would be more in-demand than further units that would require transportation to these sites, thus making them more expensive. The longitude feature measures distance East to West in the city, which is predictive of price as neighborhoods such as the Upper East Side are generally more expensive. These features and their relative importance indicate that location is likely the most important factor in unit pricing, with the size of the unit and, consequentially, the party being the second.

Figure 5

Top 15 Feature Importances



VIII. Conclusions and Recommendations

The purpose of this analysis was to create and compare regression models to predict the nightly Airbnb price in New York City. Our tuned Gradient Boosting model was able to achieve the best results, with about 47.6% of the variation in Airbnb pricing explained by the model and the predictions off by about $76.46 on average. Although these results were obtained using cross-validation and multiple model comparisons, they are still not ideal. Our models are simply limited in their predictive power given the data provided. However, our results are still useful for business recommendations despite the limitations:

I) First, all models seemed to underperform on the data provided, as indicated by their test metrics being under 0.50 (meaning they all explained less than half of the variability in price) and the highest test R^2 being 0.498. We may be able to get a more accurate picture of price were

the data to be expanded to include different predictors, such as amenities, property quality, and seasonal demand.

II) Even with the limited data, we can clearly see that location is a major predictor of price, with two out of the top three features by importance being location-based (distance to Times Square and longitude). These results indicate that there should exist a pricing strategy that is focused on location – distance to landmarks, high demand areas, etc.

III) Finally, the type of unit was also identified by the Gradient Boosting model as another important predictor of price, signaling that price does not just depend on location but also on the party size. There should be a focus on targeting these specific group sizes, such as singles, couples, families, and large groups, with pricing strategies tailored to their needs.

Another limitation to our analysis besides the overall lack of predictive power is that the data is now seven years old. Since the data was collected pre-COVID, our pricing predictions do not predict current-day pricing dynamics. Changes in the economy, travel behaviors, and demand patterns after the pandemic mean that the models learned relationships in the data that may no longer be applicable to today's Airbnb market. Ideally, the next step in this analysis would be to recollect the data, including only current data with the additional features mentioned above added. The models could then be retrained on more applicable and predictive data, likely improving their predictive accuracy.

IX. References

Dgomonov. (2019). *New York City Airbnb Open Data*. Www.kaggle.com. https://www.kaggle.com/datasets/dgomonov/new-york-city-Airbnb-open-data